Social Movements in the Digital Space

Session 3b: Halloween Special — R Crash Course

Anna Nikolina

2026-10-30

Today: Don't Be Afraid of R

R is much less scary than it looks.

Today we will learn how to:

- find our way around RStudio;
- run and understand simple R code;
- open and inspect a dataset;
- ask simple questions of the data;
- create a first visualization;
- interpret what the output means.

Goal: open R and actually do something with data.

Learning Outcomes

At the end of today, you should be able to:

1. navigate RStudio and run simple R commands;
2. understand the basic ideas of **objects, functions, vectors, dataframes, and packages**;
3. load and inspect a dataset;
4. filter, count, summarize, and visualize simple data;
5. interpret a result without making claims the data cannot support.

How Much R Do You Already Know?

Mentimeter

There is no wrong starting point.

Getting Oriented

R and RStudio Are Not the Same Thing

R

- programming language;
- statistical computing;
- data manipulation;
- visualization;
- analysis.

R and RStudio Are Not the Same Thing

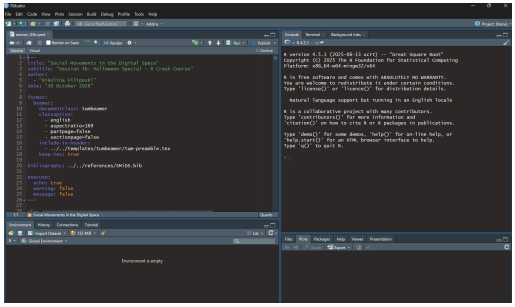
RStudio

- environment in which we work with R;
- write and save scripts;
- run code;
- inspect data;
- view plots and documentation.

R does the work. RStudio gives us a convenient place to work.

See also the official R and RStudio resources (*R*, n.d.; "RStudio IDE User Guide" 2026).

Your RStudio Workspace



The four areas you will use most:

- **Source / Script** — write and save code;
- **Console** — where R executes commands;
- **Environment** — objects currently stored in memory;
- **Files / Plots / Packages / Help** — files, output and documentation.

Scripts Are Your Record

Lines beginning with # are **comments**.

```
# This calculates the average
mean(engagement)
```

Comments help us remember:

- what the code does;
- why we did something;
- what a result means.

Do not work only in the Console. Save your code in the script.

Where Does Our Analysis Live?

For this seminar, your work should live inside the course repository.

Today we use:

```
social_movements_digital_space/
  03b_r_crash_course/
      code/
      data/
      slides/
```

Why?

- code and data stay together;
- paths can work across computers;
- analyses are easier to reproduce;
- fewer mysterious “file not found” errors.

Finding Files Reliably

Different computers store the repository in different places.

We use the `here` package to start from the repository root.

```
library(here)
```

Then:

```
here(
  "03b_r_crash_course",
  "data",
  "bluesky_climate_sample.csv"
)
```

You do not need to know where somebody else's repository is stored.

GitHub, the Repository, and `here`

For this course, the project is shared through

GitHub.

You do **not** need to learn Git today.

- GitHub gives us the shared course files and you work with your local copy of the repository;
- keep the folder structure unchanged;
- `here()` helps R find files inside that structure.

See the separate **GitHub course guide** for setup and updating the repository.

Why Do We Use `here()`?

Your repository may live somewhere different from mine.

```
C:/Users/Student/Documents/social_movements_digital_space/
```

or:

```
C:/Users/Nikolina/Documents/social_movements_digital_space/
```

But inside the repository, we share the same structure:

```
03b_r_crash_course/data/bluesky_climate_sample.csv
```

First R Commands

R as a Calculator

```
1 + 1
```

```
[1] 2
```

```
10 / 2
```

```
[1] 5
```

```
7 * 6
```

```
[1] 42
```

```
(10 + 5) / 3
```

```
[1] 5
```

Try it yourself: change one number and run the command.

Objects: Giving Things Names

We often want to store something and use it later.

```
pumpkins <- 13
ghosts <- 4

pumpkins + ghosts
```

```
[1] 17
```

Read:

```
pumpkins <- 13
```

as:

> *store 13 under the name* `pumpkins`

Objects you create will appear in the **Environment** pane.

Vectors: More Than One Value

A vector contains several values.

```
engagement <- c(3, 8, 2, 15, 5)
engagement
```

```
[1]  3  8  2 15  5
```

```
mean(engagement)
```

```
[1] 6.6
```

```
median(engagement)
```

```
[1] 5
```

```
max(engagement)
```

```
[1] 15
```

Statistics Flashback: Mean and Median

Two common ways to describe the **center** of numeric data:

Mean

Add all values and divide by the number of values.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

Statistics Flashback: Mean and Median

Median

Sort the values and take the **middle value**.
For:

$$2,\ 3,\ 5,\ 8,\ 15$$

$$\text{Mean} = 6.6 \qquad \text{Median} = 5$$

Both describe the center — but in different ways.

Mean or Median?

The **mean** uses every value. The **median** depends on the middle position. Consider:

$$2,\ 3,\ 5,\ 8,\ 15$$

Now replace the largest value:

$$2,\ 3,\ 5,\ 8,\ \mathbf{100}$$

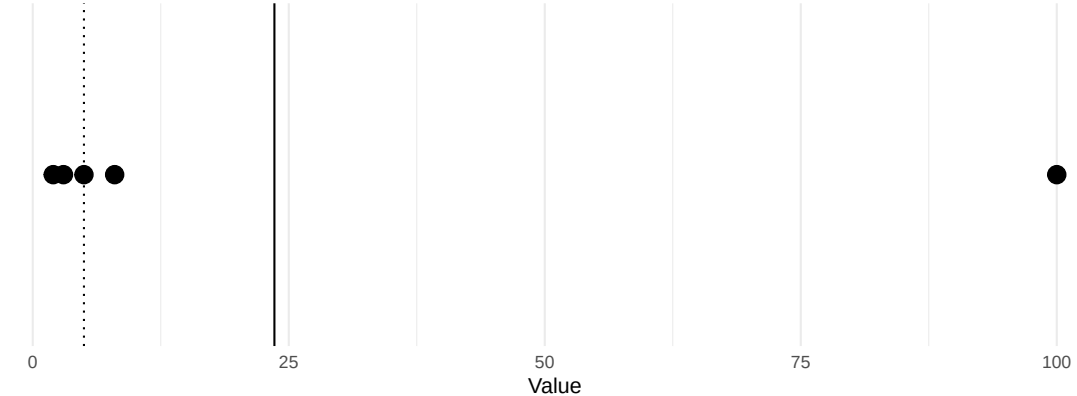
$$\text{Mean} = 23.6 \qquad \text{Median} = 5$$

The extreme value strongly changes the **mean**. The **median** is much less affected.

If mean and median differ strongly, look at the distribution.

Mean, Median, and the Distribution

One extreme value can pull the mean



Solid line: mean **Dotted line:** median

How to Read a Function

A useful pattern is:

function(input)

For example:

```
mean(engagement)
```

```
[1] 6.6
```

means:

Take `engagement` and calculate its mean.

Later, functions can have additional arguments:

```
mean(x, na.rm = TRUE)
```

You do not need to worry about all of those yet.

Objects, Vectors, Dataframes, Functions

Object something stored under a name

Vector a collection of values

Dataframe a table of observations and variables

Function an instruction that does something

Packages and Help

What Is a Package?

R already contains many functions.

A **package** adds additional tools.

For this seminar, we will often use:

```
library(tidyverse)
library(here)
```

Two different actions:

```
install.packages("tidyverse")
```

Install once on your computer.

```
library(tidyverse)
```

Load again when you start a new R session.

Where Do I Look Things Up?

You are **not expected to memorize R**.

Useful places to look:

- `?function_name`
- RStudio Help pane
- package documentation
- Posit cheatsheets
- course starter and solution scripts
- documentation / search

For example:

```
?mean
```

A large part of programming is knowing **what to look up**.

Errors Are Normal

At some point today, you will get an error.

It does **not** mean:

- your entire analysis is lost;
- your computer is broken;
- you need to start everything again.

It means:

R could not carry out the instruction you gave it.

First steps:

1. read the error message;
2. check the line that caused it;
3. check names, brackets, commas, and paths.

From R to Data

From Individual Values to a Dataset

So far:

numbers → objects → vectors → functions

Now we move to:

a dataframe

Our teaching example uses public posts from **Bluesky**.

What Is Bluesky?

For today, you only need to know:

- Bluesky is a social-media platform;
- users can publish public posts and replies;
- those activities leave digital traces;
- some of those traces can be represented as data.

The important methodological question is:

What does one row in our dataset actually represent?

Climate Communication on Bluesky



Our Teaching Dataset

Our prepared dataset contains:

1,000 English-language climate-related Bluesky posts

Each **row** represents one post. Each **column** describes something about that post.

Variables include:

- `created_at`
- `text`
- `language`
- `has_images`
- `is_reply`
- `text_length`
- `has_hashtag`
- `has_link`

A Small Methodological Warning

Posts entered the teaching dataset through a **climate-related keyword search**.

That means:

- the sample contains posts matching our retrieval rule;
- it is useful for learning and descriptive analysis;
- it is **not** a representative sample of all climate communication;
- it is **not** a representative sample of Bluesky.

How data were collected matters for what we can claim.

Rows and Columns

Row

One **observation**. In today's dataset: **one Bluesky post**

Column

One **variable** describing those observations. For example:

- date;
- text;
- reply status;
- image status;
- text length.

This distinction will matter throughout the semester.

Working With the Dataset

Step 1: Load the Data

First load the packages:

```
library(tidyverse)
library(here)
```

Then load the CSV:

```
posts <- read_csv(
  here(
    "03b_r_crash_course",
    "data",
    "bluesky_climate_sample.csv"
  )
)
```

Did R actually load what I expected?

Step 2: Inspect Before You Analyze

Useful first commands:

```
View(posts)
head(posts)
names(posts)
nrow(posts)
glimpse(posts)
```

Before doing any analysis, ask:

- What does one row represent?
- What variables are available?
- How many observations do we have?
- What type of information does each variable contain?

What Did We Load?

```
nrow(posts)
```

```
[1] 1000
```

```
names(posts)
```

```
[1] "post_id"     "created_at"  "text"         "language"    "has_images"
[6] "is_reply"    "text_length" "has_hashtag" "has_link"
```

Our dataframe has:

- **1,000 observations**
- **9 variables**

Always inspect first. Analyze second.

What Kinds of Values Do We Have?

When we use `glimpse(posts)`, R tells us what kind of information each variable contains.

```
glimpse(posts)
```

Some common types in today's dataset:

- `<chr>` — **character**: text, such as a post;
- `<dbl>` — **numeric**: numbers, such as `text_length`;
- `<lgl>` — **logical**: `TRUE` or `FALSE`;
- `<dttm>` — **date and time**.

Variables can store different kinds of information.

The type affects what we can sensibly do with a variable.

Step 3: Count Categories

Suppose we want to know how many posts are replies.

```
posts |>
  count(is_reply)
```

```
# A tibble: 2 x 2
  is_reply     n
  <lgl>    <int>
1 FALSE      756
2 TRUE       244
```

Read the pipe |> approximately as:

“and then”

Take `posts` **and then** count observations by reply status.

Try Another Count

How many posts contain images?

```
posts |>
  count(has_images)
```

```
# A tibble: 2 x 2
  has_images     n
  <lgl>      <int>
1 FALSE        805
2 TRUE         195
```

Try Another Count

And links?

```
posts |>
  count(has_link)
```

```
# A tibble: 2 x 2
  has_link     n
  <lgl>    <int>
1 FALSE      758
2 TRUE       242
```

Same basic operation.

Different question.

Step 4: Filter Observations

Sometimes we want only some rows.

For example, keep only replies:

```
reply_posts <- posts |>
  filter(is_reply == TRUE)
```

Then:

```
head(reply_posts)
```

This means:

Take `posts`, keep rows where `is_reply` is TRUE, and save them as `reply_posts`.

<- and == Are Different

Two small symbols with very different jobs:

```
<-    stores something

==    asks whether two things are equal
```

For example:

```
reply_posts <- posts |>
  filter(is_reply == TRUE)
```

Read it approximately as:

Create `reply_posts` by keeping the rows where `is_reply` equals `TRUE`.

Small symbols can change what R does.

Step 5: Calculate a Statistic

How long are the posts?

```
mean(posts$text_length)
```

```
[1] 210.232
```

```
median(posts$text_length)
```

```
[1] 229
```

- What does the number summarize?
- Is the mean similar to the median?
- What does the result tell us?
- What does it **not** tell us?

Descriptive statistics summarize what is in our data.

What Does `summarise()` Do?

`summarise()` turns many observations into one or more **summary values**.

For example:

1,000 posts

↓

mean text length

median text length

With `group_by()`, we calculate those summaries **separately for each group**.

Step 6: Summarize Groups

We can combine operations.

```
posts |>
  group_by(is_reply) |>
  summarise(
    n_posts = n(),
    mean_length = mean(text_length),
    median_length = median(text_length)
  )
```

```
# A tibble: 2 x 4
  is_reply n_posts mean_length median_length
  <lgl>      <int>       <dbl>         <dbl>
1 FALSE        756        209.           227
2 TRUE         244        214.           236
```

How to Read a ggplot

A basic plot asks three questions:

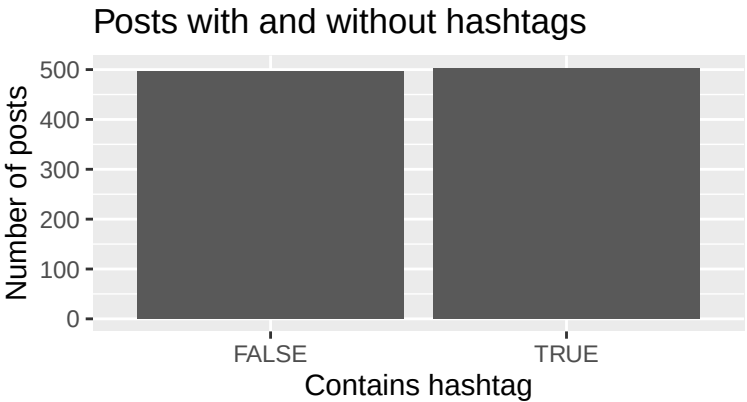
What data? Which variables? What kind of visual?

```
ggplot(
  data,
  aes(
    x = ...,
    y = ...
  )
) +
  geom_...()
```

Step 7: Make a First Plot

```
posts |>
  count(has_hashtag) |>
  ggplot(
    aes(
      x = has_hashtag,
      y = n
    )
  ) +
  geom_col() +
  labs(
    title = "Posts with and without hashtags",
    x = "Contains hashtag",
    y = "Number of posts"
  )
```

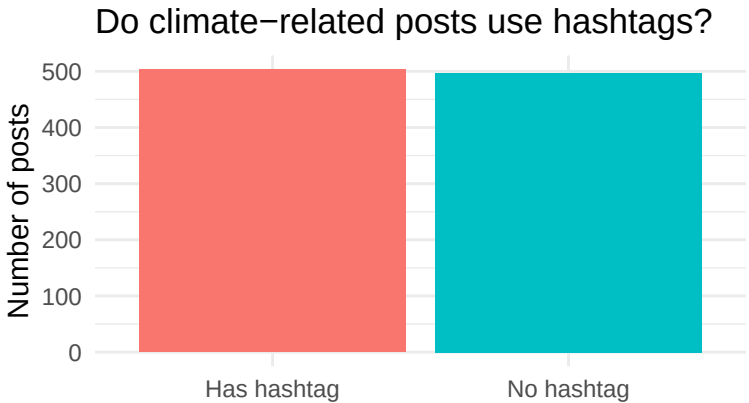
Step 7: Make a First Plot



One Small Step Prettier

```
posts |>
  count(has_hashtag) |>
  mutate(
    hashtag = if_else(
      has_hashtag,
      "Has hashtag",
      "No hashtag"
    )
  ) |>
  ggplot(
    aes(
      x = hashtag,
      y = n,
      fill = hashtag
    )
  ) +
  geom_col(
    show.legend = FALSE
  ) +
  labs(
    title = "Do climate-related posts use hashtags?",
    x = NULL,
    y = "Number of posts"
  ) +
  theme_minimal()
```

One Small Step Prettier

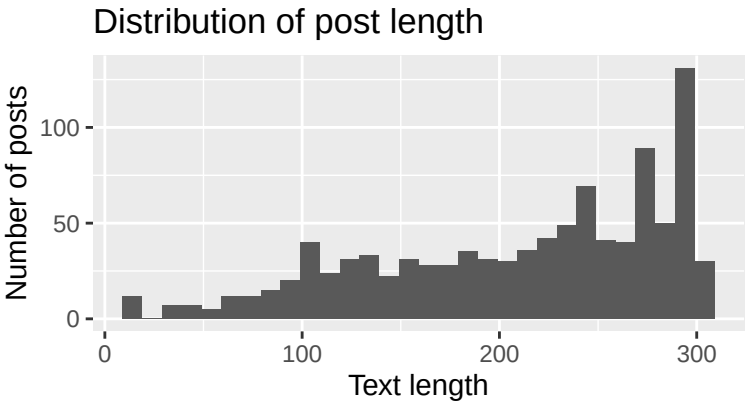


Another Kind of Plot

A histogram shows the distribution of a numeric variable.

```
ggplot(
  posts,
  aes(
    x = text_length
  )
) +
  geom_histogram() +
  labs(
    title = "Distribution of post length",
    x = "Text length",
    y = "Number of posts"
  )
```

Another Kind of Plot



Practice

Halloween R Scavenger Hunt

Enter the Data Crypt

Working first alone, then together:

1. How many observations are in the dataset?
2. Which variables are available?
3. How many posts are replies?
4. How many posts contain an image?
5. How many posts contain a link?
6. What is the median text length?
7. Are reply posts longer or shorter on average?
8. Create one simple plot.

Halloween R Scavenger Hunt

When you are finished, write **one careful substantive sentence**.

For example:
"In this teaching sample, ..."

Do **not** write:
"This proves that ..."

Be ready to explain:

- what you calculated;
- what your output shows;
- what you cannot conclude from it.

From Output to Interpretation

R gives us output.

Our job is to decide what it means.

A careful sentence:
"In this teaching sample, posts with X occur more frequently than posts with Y."

A much stronger claim:
"This proves that X is more common on Bluesky."

Why is the second statement too strong?

What Can These Data Not Tell Us?

A digital-trace dataset can show us some forms of observable activity.

It does not automatically reveal:

- why somebody posted;
- who actually saw the post;
- people who read but did not interact;
- private communication;
- offline participation;
- effects of recommendation algorithms;
- whether our sample represents a whole movement or platform.

Data are traces of activity — not the activity itself.

Where to Go From Here

Useful R Resources

Learning R

- **R for Data Science** — a broad introduction to working with data in R and the tidyverse (Wickham and Grolemund 2017).

Quick Help

- **Posit Cheatsheets** — quick references for commonly used R tools and packages ("Cheatsheets" 2025).
- **The R Graph Gallery** — reproducible examples of many different visualization types (Holtz, n.d.).
- **RStudio** — documentation and information about the development environment ("RStudio IDE User Guide" 2026).

Which Resource Should I Use?

I forgot a command → **Posit Cheatsheets**

I want to understand the workflow → **R for Data Science**

I want to make a graph → **R Graph Gallery**

Looking things up is part of programming.

Wrap-up

R Survival Checklist

- ☐ I can open the course project.
- ☐ I can run a line of R code.
- ☐ I can create an object.
- ☐ I understand what a function does.
- ☐ I can load a CSV.
- ☐ I understand rows and variables.
- ☐ I can recognize basic variable types.
- ☐ I can inspect a dataframe.
- ☐ I can filter observations.
- ☐ I can count categories.
- ☐ I can calculate a simple statistic.
- ☐ I can summarise groups.
- ☐ I can make a basic plot.
- ☐ I can interpret a result cautiously.

When R Does Not Work

Before worrying, check:

1. Did I run the line above?
2. Did I spell the object or variable correctly?
3. Are brackets and quotation marks closed?
4. Did I load the package?
5. Is the file where I expect it to be?
6. Did I read the error message?

When R Does Not Work

If you see:

```
object 'something' not found
```

ask:

Where was `something` supposed to be created?

Then:

- check the course starter script;
- check the Help pane / documentation;
- compare with the solution script;
- ask somebody.

Debugging is part of programming.

Before We Finish

Mentimeter

Take-Away

You do **not** need to know every R function.

You need to understand the workflow:

Question → Data → Code → Result → Interpretation

And when you get stuck:

Read → Check → Look it up → Try again

See you in Session 4.

References I

"Cheatsheets." 2025. In *Posit Open Source*. https://opensource.posit.co/resources/cheatsheets/.

Holtz, Yan. n.d. "The R Graph Gallery – Help and Inspiration for R Charts." In *The R Graph Gallery*. Accessed September 15, 2026. https://r-graph-gallery.com/?utm_source=chatgpt.com.

R: The R Project for Statistical Computing. n.d. Accessed September 15, 2026. https://www.r-project.org/?utm_source=chatgpt.com.

References II

"RStudio IDE User Guide." 2026. In *RStudio User Guide*. https://docs.posit.co/ide/user/#rstudio-ide-oss-downloads?ref=rstudio-legacy.

Wickham, Hadley, and Garrett Grolemund. 2017. *R for Data Science*. Vol. 2. O'reilly Sebastopol. https://www.jstatsoft.org/htaccess.php?volume=077&type=b&issue=01&filename=paper.